

Counting Trees Without Listing Them

A machine-checked proof of Kirchhoff’s matrix-tree theorem in Lean 4

Carles Marín*

Independent researcher
karlesmarin@gmail.com

June 2026

Abstract

I present a machine-checked, *sorry*-free proof, in Lean 4 over Mathlib, of *Kirchhoff’s matrix-tree theorem*: over any integral domain, the determinant of the reduced Laplacian of a finite simple graph equals its number of spanning trees. The proof is assembled from four formalized pieces, each of independent use: the Cauchy–Binet formula for $\det(AB)$ of a rectangular pair; the *oriented* incidence matrix and its Gram factorization $NN^T = D - A$ (Mathlib has only the unoriented incidence matrix, whose Gram matrix is $D + A$); the Cauchy–Binet expansion of the reduced Laplacian determinant into a sum of squared maximal minors; and a dichotomy theorem identifying the squared minor as 1 exactly on the edge sets of spanning trees and 0 elsewhere. The dichotomy is proved *without* the textbook leaf-deletion induction: each non-root vertex of a connected subgraph is assigned its *parent edge* on a geodesic to the root, the resulting recolumned minor is upper-triangular under the lexicographic key (distance to root, vertex), and the determinant is the product of its ± 1 diagonal. A by-product of the design is that no separate acyclicity argument is ever constructed: connectivity and the edge count—which together, classically, already characterize trees—are the only inputs the formal proof consumes. The headline theorem depends only on the three standard axioms. I am exact about the boundary: the result is for finite simple graphs over an integral domain; the weighted Kirchhoff polynomial, the all-minors theorem and the directed version are mapped but not built; an independent Lean formalization of Cauchy–Binet (not in Mathlib) appeared in the Grinberg-textbook project and is discussed in Section 9; for the matrix-tree theorem itself I was unable to locate a prior formalization in any proof assistant, though such a claim can only ever be made to the best of one’s knowledge.

Reader’s map. If you read only one thing, read Theorem 1 and Figure 6—the statement, and the picture of why a spanning tree’s minor is ± 1 . The proof is a single chain (Figure 3); every link has its own section, a figure or a worked table, and an example small enough to check by hand. The honest limits are in Section 9.

1 Introduction: two memoirs and a circuit

On the 30th of November, 1812, two memoirs on determinants were read at the Institut de France on the same day: one by Jacques Binet, one by Augustin-Louis Cauchy [1, 2]. Both contained, among much else, the formula now bearing both names: the determinant of a product of two rectangular

*Formalized with AI assistance (Claude, Anthropic); the mathematics and all claims are the author’s responsibility. The methodology is discussed in Section 10.

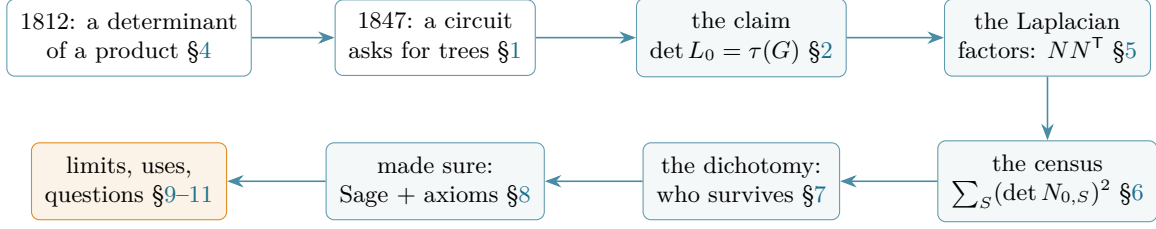


Figure 1: The reader’s map: the themes of the paper in their logical order—two classical ingredients, the claim, the factorization that decomposes it, the census, the decision, the checks, and what remains.

matrices is a sum, over all ways of selecting a square from the shared dimension, of products of corresponding minors. It is the earliest theorem in this paper, and the deepest single ingredient.

Thirty-five years later, Gustav Kirchhoff was not studying determinants. He was studying electrical circuits—the linear equations governing currents in a resistor network [3]—and found that the common denominator of their solution, the determinant that Cramer’s rule places under every current, counts something purely combinatorial: the spanning trees of the network’s graph. In modern dress, with D the diagonal of vertex degrees and A the adjacency matrix, the *Laplacian* $L = D - A$ of a graph G on n vertices is singular (its rows sum to zero), but striking any one row and the matching column leaves an $(n - 1) \times (n - 1)$ matrix L_0 with

$$\det L_0 = \tau(G),$$

the number of spanning trees of G —whichever row was struck. A determinant computable in polynomial time counts a family that is typically exponential; the matrix knows the answer without anyone listing a single tree.

The theorem then led a double life. Borchardt, in 1860, was the first to obtain the labelled-tree count n^{n-2} by evaluating a determinant [4]; Cayley’s short note of 1889, which gave the formula its name and credits Borchardt, exhibits the count that the matrix-tree determinant produces on the complete graph [5]. In 1940, Brooks, Smith, Stone and Tutte dissected rectangles into squares by interpreting the dissection as a current flowing through a planar network—Kirchhoff’s circuits returning to mathematics [6]. Tutte extended the count to directed graphs and arborescences [7]; Chaiken and Kleitman proved the all-minors generalization [8, 9]. In modern probability the theorem is load-bearing: Wilson’s algorithm samples a uniform spanning tree faster than the cover time [14], the transfer-current theorem makes the uniform spanning tree a determinantal process [15], and effective resistances—ratios of spanning tree counts—drive spectral sparsification [16]. The standard accounts are [10, 11, 12, 13, 17].

What the theorem did not have, as far as I could find, is a machine-checked proof. The Lean mathematical library [20] has the Laplacian matrix and knows its kernel; it has the *unoriented* incidence matrix; it recently gained, in an independent textbook-formalization project, the Cauchy–Binet formula [18, 19]—but not the matrix-tree theorem, and I found no formalization of it in Isabelle’s Archive of Formal Proofs or in the Coq/Rocq ecosystem either. This paper closes that gap, as Part V of a series formalizing the walk–matching–spectrum circle of ideas around the matching polynomial [21, 22, 23, 24, 25].

One more thing, and it is the twist this paper sets down. Every textbook proof of the dichotomy at the heart of the theorem—*a set of $n - 1$ edges has minor ± 1 if it is a spanning tree and 0 otherwise*—peels a leaf and inducts. The formal proof here does something a machine finds far more comfortable: it *sorts*. Each non-root vertex is matched to its *parent edge* on a geodesic

toward the root; ordering vertices by distance makes the minor upper-triangular with ± 1 diagonal, and the determinant falls out as a product (Figure 6). No induction on graphs anywhere. A smaller observation, of proof engineering rather than mathematics: of the two halves of the tree characterization “connected and acyclic with $n - 1$ edges”, the formal proof only ever consumes connectivity plus the count—the equivalent acyclicity half, though of course implied, never has to be materialized as a lemma. The machine is never asked to look for a cycle.

2 The result

Throughout, G is a finite simple graph on a vertex type V with a linear order (any enumeration of the vertices), $v_0 \in V$ a fixed *root*, and R a commutative ring—an integral domain where stated. Write $n = |V|$. The Lean statement of the headline theorem is:

Theorem 1 (`SimpleGraph.det_reducedLapMatrix_eq_card_spanningTrees`). *Let R be an integral domain. Then*

$$\det L_0 = \# \left\{ S \subseteq \text{Sym}^2 V : |S| = n - 1, S \subseteq E(G), (V, S) \text{ is a tree} \right\}$$

where L_0 is the Laplacian of G with the row and column of v_0 deleted, and the count is cast into R .

In Lean the right-hand side is a `Finset.filter` over the subtype of $(n - 1)$ -element subsets of $\text{Sym}^2 V$, the condition being $\uparrow S \subseteq G.\text{edgeSet} \wedge (\text{fromEdgeSet } \uparrow S).\text{IsTree}$; since `fromEdgeSet` places the subgraph on *all* of V , `IsTree` already encodes spanning. Every spanning tree of G has exactly $n - 1$ edges and is determined by its edge set, so the filter counts spanning trees, each exactly once.

What is the theorem *for*? It converts an exponential enumeration into a polynomial-time linear-algebra computation. The Petersen graph has 2000 spanning trees; nobody finds them by listing. One writes down the 9×9 reduced Laplacian and takes a determinant. The same trade powers everything in Section 10: reliability counts, uniform sampling, effective resistances. Table 1 shows the theorem in action on six classical graphs, with both sides computed independently in SageMath.

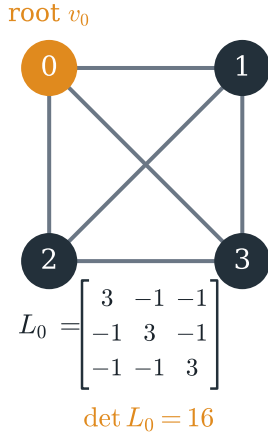
graph	n	$\det L_0$	$\tau(G)$ (independent count)
K_4	4	16	16
K_5	5	125	125
C_6 (cycle)	6	6	6
$K_{3,3}$	6	81	81
Q_3 (cube)	8	384	384
Petersen	10	2000	2000

Table 1: Both sides of Theorem 1 computed independently (SageMath, exact integer arithmetic). Note K_4 and K_5 are Cayley’s 4^2 and 5^3 ; C_6 counts its six single-edge deletions.

3 The chain in four stones

The formal proof is a chain of four results, each usable on its own. Figure 3 shows the dependencies; Table 2 lists the Lean names. The narrative of the next four sections simply walks the chain.

K_4 and its reduced Laplacian



the sixteen spanning trees that $\det L_0$ counts

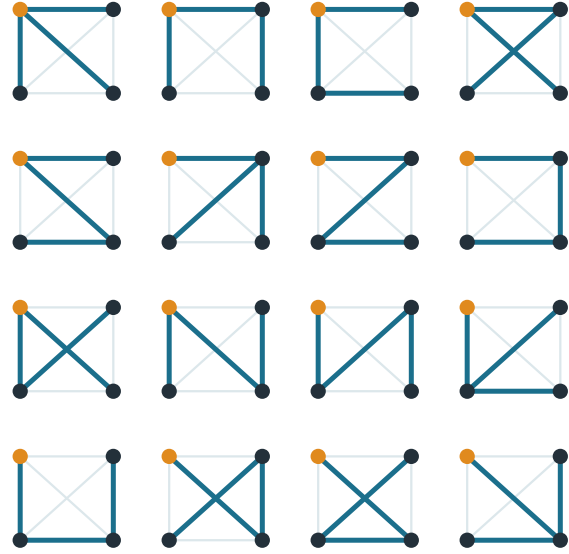


Figure 2: The theorem on K_4 , rooted at v_0 . The reduced Laplacian is the 3×3 matrix on the left; its determinant is 16, and on the right are the sixteen spanning trees it counts. Cayley’s formula n^{n-2} gives $4^2 = 16$; the determinant arrives at the same number without enumerating anything.

Lean theorem	statement	file (under Ihara/)
<code>det_mul_cauchyBinet</code>	$\det(AB) = \sum_S \det A_S \det B_S$	<code>CauchyBinet.lean</code>
<code>orientedIncMatrix_mul_transpose</code>	$NN^T = D - A$	<code>OrientedIncidence.lean</code>
<code>det_reducedLapMatrix_eq_sum_sq</code>	$\det L_0 = \sum_S (\det N_{0,S})^2$	<code>MatrixTree.lean</code>
<code>sq_det_minor_eq_ite</code>	$(\det N_{0,S})^2 = \mathbb{I}[S \text{ spans a tree}]$	<code>SpanningTreeMinor.lean</code>
<code>det_reducedLapMatrix_eq_card_spanningTrees</code>	$\det L_0 = \tau(G)$	<code>Kirchhoff.lean</code>

Table 2: The load-bearing results. All are `sorry-free`; each `#print axioms` reports exactly `propext`, `Classical.choice`, `Quot.sound`.

4 A theorem from 1812

Theorem 2 (`Matrix.det_mul_cauchyBinet`). *Let R be a commutative ring, A an $m \times n$ and B an $n \times m$ matrix over R . Then*

$$\det(AB) = \sum_{\substack{S \subseteq \{1, \dots, n\} \\ |S|=m}} \det A_{\cdot, S} \cdot \det B_{S, \cdot}$$

where $A_{\cdot, S}$ keeps the columns of A indexed by S in increasing order, and $B_{S, \cdot}$ the corresponding rows of B .

If $m > n$ the sum is empty and the determinant is 0; if $m = n$ it is the multiplicativity of $\det$. The genuinely rectangular case is the useful one: *every Gram matrix is a product of a matrix with its transpose*, and Cauchy–Binet turns its determinant into a sum of squares—nonnegativity for

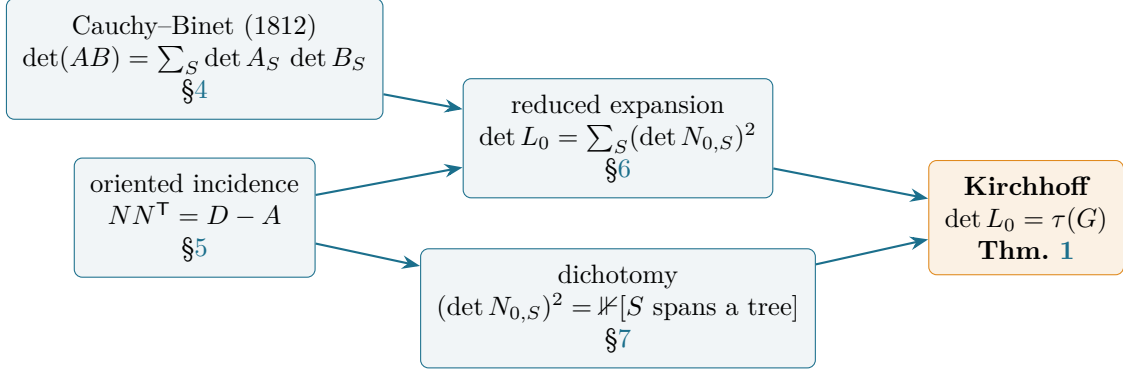


Figure 3: The proof chain. Two classical ingredients combine into the sum-of-squares expansion; the dichotomy decides each summand; the count falls out.

$$\begin{array}{ccc}
 S = \{1, 2\} & S = \{1, 3\} & S = \{2, 3\} \\
 \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline \end{array} \times \begin{array}{|c|c|} \hline 7 & 1 \\ \hline 2 & 8 \\ \hline 3 & 4 \\ \hline \end{array} & \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline \end{array} \times \begin{array}{|c|c|} \hline 7 & 1 \\ \hline 2 & 8 \\ \hline 3 & 4 \\ \hline \end{array} & \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline \end{array} \times \begin{array}{|c|c|} \hline 7 & 1 \\ \hline 2 & 8 \\ \hline 3 & 4 \\ \hline \end{array} \\
 \det A_S \cdot \det B_S = (-3) \cdot (54) = -162 & \det A_S \cdot \det B_S = (-6) \cdot (25) = -150 & \det A_S \cdot \det B_S = (-3) \cdot (-16) = 48
 \end{array}$$

$$\det(AB) = -162 - 150 + 48 = -264 : \text{one product of minors per choice of columns}$$

Figure 4: Cauchy–Binet for a 2×3 times 3×2 product. Three ways to choose two of the three middle indices; each contributes the product of the two shaded minors. The three products $-162 - 150 + 48$ sum to $\det(AB) = -264$, which one checks directly: $AB = \begin{pmatrix} 20 & 29 \\ 56 & 68 \end{pmatrix}$.

free over the reals, and, in this paper, a sum over candidate edge sets. Figure 4 works the smallest nontrivial case by hand.

The Lean proof has three moves, and the middle one is where the formula’s content lives. *First*, expanding $\det(AB)$ by the Leibniz formula and distributing the product over the matrix-product sums gives an expansion over *all* functions $g : \{1..m\} \rightarrow \{1..n\}$,

$$\det(AB) = \sum_g \det A_{.,g} \cdot \prod_i B_{g(i),i} \quad (\text{det_mul_eq_sum_submatrix}),$$

where $A_{.,g}$ repeats columns as g does. *Second*—the heart—fix an injective image S and sum over the $m!$ relabellings π of its columns: the A -minor picks up $\text{sgn } \pi$, and summing the plain products $\prod_i B$ against that sign *reassembles the determinant of B_S* . out of nothing but the Leibniz formula read backwards (**fiber_sum**). *Third*, non-injective g die (a repeated column), and the injective ones biject with pairs (subset, relabelling); Lean’s **Finset.sum_bij** does the bookkeeping. Nothing here is original mathematics—the proof is essentially Binet’s—but the care in the middle step is what a proof assistant buys: the sign bookkeeping is checked once, completely, and never again by anyone.

Mathlib does not contain Cauchy–Binet. An independent formalization appeared recently in the Lean project accompanying Grinberg’s textbook [18, 19], with a different architecture; Section 9 compares. The version here is self-contained over a commutative ring and stated against Mathlib’s

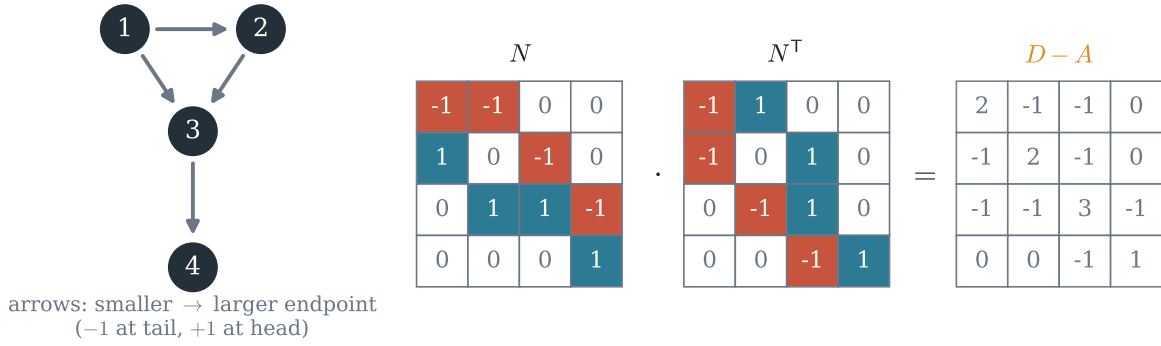


Figure 5: Theorem 3 on the triangle-with-tail graph. Each column of N holds one edge's ± 1 pair; row u dotted with itself counts $(\pm 1)^2$ once per incident edge (the degree), row u dotted with row w sees only the shared edge, contributing -1 . The signs are what make the Laplacian: the unoriented matrix would give $D + A$.

Matrix API, with the sorted-subset indexing (`Finset.orderEmbOffFin`) that the rest of the chain consumes.

5 The Laplacian is a Gram matrix

Mathlib has an incidence matrix, but the unoriented one: a $V \times \text{Sym}^2 V$ array of 0s and 1s. Its Gram matrix is $D + A$, the *signless* Laplacian—the wrong sign for counting trees. The missing object is the *oriented* incidence matrix, and it is the one place in this development where a definition had to be designed rather than found.

Definition 1 (`SimpleGraph.orientedIncMatrix`). *For a linearly ordered vertex type, let $N \in R^{V \times \text{Sym}^2 V}$ have, in the column of an edge $e = \{u, w\}$ with $u < w$: entry $+1$ at row w (the larger endpoint), -1 at row u , and 0 elsewhere; columns of non-edges are zero.*

Orienting every edge from its smaller to its larger endpoint is arbitrary, and harmlessly so: any orientation gives the same Gram matrix, because each edge contributes $(\pm 1)^2$ to a diagonal entry and $(+1)(-1)$ to an off-diagonal one. That computation is the whole proof of:

Theorem 3 (`SimpleGraph.orientedIncMatrix_mul_transpose`). $N N^T = D - A = L.$

The utility of the factorization is that it moves spectral facts to bookkeeping: positive semidefiniteness of L is immediate ($x^T N N^T x = \|N^T x\|^2$ over $\mathbb{R}$), the kernel is the locally-constant functions, and—the use made here—the determinant of any square submatrix of L becomes a Cauchy–Binet sum. In Lean the proof is an entrywise computation: the diagonal squares to the unoriented incidence row (`orientedIncMatrix_mul_self`), whose sum Mathlib already knows is the degree; the off-diagonal product of two distinct rows is supported on their common edge, where the signs are opposite by construction.

6 Delete a root, square the minors

The Laplacian itself is singular, so the count lives one deletion down. Fix the root v_0 ; write N_0 for N without the row of v_0 , and L_0 for L without the row and column of v_0 . Restricting Theorem 3 gives $L_0 = N_0 N_0^\top$, and now N_0 is an $(n-1) \times \binom{|V|}{2}$ rectangle: exactly Cauchy–Binet’s shape, with $B = N_0^\top$ the transpose of $A = N_0$, so each summand is a product of a minor with its own transpose—a square.

Theorem 4 (`SimpleGraph.det_reducedLapMatrix_eq_sum_sq`).

$$\det L_0 = \sum_{\substack{S \subseteq \text{Sym}^2 V \\ |S|=n-1}} (\det N_{0,S})^2$$

where $N_{0,S}$ is the square minor of N_0 on the columns S .

Note where the sum ranges: over *all* $(n-1)$ -subsets of vertex pairs, edges or not. The columns of non-edges are zero, so their subsets contribute nothing—but they are honestly in the sum, and the formal statement says so. For K_4 the sum has $\binom{6}{3} = 20$ terms; Table 3 classifies them. The determinant has become a *census*: the rest of the proof is deciding, subset by subset, who contributes.

the 20 subsets (K_4 , $ S = 3$)	how many	$(\det N_{0,S})^2$	contribution
spanning trees	16	1	16
triangles (4th vertex isolated)	4	0	0

Table 3: The census for K_4 : every 3-subset of its 6 edges either spans a tree or is one of the four triangles, which leave a vertex stranded. $\det L_0 = 16 + 0 = 16$, matching Figure 2. (A 3-subset of edges on 4 vertices fails to be a spanning tree exactly when it is a triangle.)

One technical remark, since it is the kind a formalization cannot wave away: Cauchy–Binet needs a linear order on the column type $\text{Sym}^2 V$ to name its sorted minors. None is canonical, so the development installs a scoped one—lexicographic on the (smaller, larger) endpoint pair—purely as indexing apparatus. Determinants squared do not care which order was chosen, and the statement of Theorem 4 quantifies over subsets, not orderings.

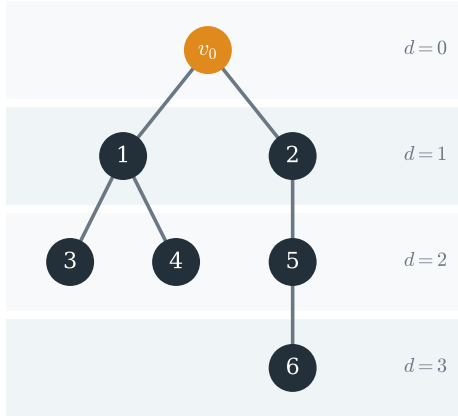
7 Which minors survive: sorting instead of inducting

Theorem 5 (`SimpleGraph.sq_det_minor_eq_ite`). *Let R be an integral domain and $|S| = n-1$. Then*

$$(\det N_{0,S})^2 = \begin{cases} 1 & \text{if } S \subseteq E(G) \text{ and } (V, S) \text{ is connected,} \\ 0 & \text{otherwise.} \end{cases}$$

With $n-1$ edges, “connected” is “spanning tree”; the Lean statement uses the connectivity form because that is what the proof consumes, and a separate lemma converts it to `IsTree` for the headline (`connected_fromEdgeSet_iff_isTree`). The proof is three cases, in increasing order of interest.

a spanning tree, rooted; key = (distance, vertex)



	$\{0, 1\}$ e_1	$\{0, 2\}$ e_2	$\{1, 3\}$ e_3	$\{1, 4\}$ e_4	$\{2, 5\}$ e_5	$\{5, 6\}$ e_6
1	1	0	-1	-1	0	0
2	0	1	0	0	-1	0
3	0	0	1	0	0	0
4	0	0	0	1	0	0
5	0	0	0	0	1	-1
6	0	0	0	0	0	1

rows: vertices by key $\uparrow$ · columns: their parent edges
zeros below the diagonal, ± 1 on it $\Rightarrow \det = \pm 1$

Figure 6: The connected case of Theorem 5. Left: a spanning tree rooted at v_0 , vertices keyed by (distance to root, vertex). Right: the minor $N_{0,S}$ with each vertex's column chosen as its *parent edge* and both sides sorted by the key. A vertex meets its own parent edge on the diagonal (± 1 , framed) and its children's parent edges to the right (their key is larger); everything below the diagonal vanishes, so $\det = \pm 1$ by reading the diagonal.

A non-edge in S . Its column of N_0 is zero, and a determinant with a zero column is zero. This is also where the diagonal of $\text{Sym}^2 V$ goes: a loop $\{v, v\}$ is not an edge of a simple graph, so subsets containing one die here too.

All edges, but (V, S) disconnected. Some connected component C misses the root. Add up the rows of $N_{0,S}$ over the vertices of C : each column of an edge inside C meets both its $+1$ and its -1 ; no edge of S leaves C ; the result is the zero row vector, exhibited in Lean as a nonzero kernel vector of the transpose (`Matrix.exists_vecMul_eq_zero_iff`—this is the one step that wants an integral domain). For the failing subsets of Table 3 the component is a single stranded vertex, and the dependence is just “its row is zero”; in general C can be larger, and the indicator vector of C does the work.

All edges, (V, S) connected. This is the case the textbooks prove by peeling a leaf and inducting on n , and the case where the formal proof does something different.

Every non-root vertex u of a connected subgraph has a neighbour strictly closer to the root: the second vertex of any geodesic from u to v_0 . Call the edge to that neighbour the *parent edge* of u . Two facts, both proved by distance arithmetic alone:

- *The map $u \mapsto \text{parent edge of } u$ is injective.* If two vertices shared a parent edge in the swapped way (u the parent of u' and u' of u), their distances to the root would satisfy $d(u) = d(u') + 2$. With $|S| = n - 1$ and $n - 1$ non-root vertices, injective means bijective: *every* edge of S is somebody's parent edge.
- *Triangularity.* Recolumn the minor so that vertex u 's column is its parent edge, and sort rows and columns by the key $(\text{dist}(u, v_0), u)$. A nonzero entry in row u , column (parent edge of w) forces $u \in \{w, \text{parent of } w\}$: either the diagonal, or $\text{dist}(u, v_0) = \text{dist}(w, v_0) - 1$, strictly *smaller* key. Everything below the diagonal is zero (Figure 6), the diagonal entries are ± 1 (each vertex sits on its own parent edge), and $\det N_{0,S} = \pm 1$.

In Lean the recolumning is an `Equiv` built from injectivity plus the cardinality count, the sorting is `Finset.orderIsoOfFin` applied to the image of the key, the vanishing is `Matrix.BlockTriangular` with `Matrix.det_of_upperTriangular`, and the sign washes out in the square. Two engineering choices are worth naming plainly. *No induction*: the linear order on keys does the work that leaf-deletion recursion does in the textbook proof, and a sorted matrix is something a proof assistant handles far more gracefully than a graph surgery. *No separate acyclicity lemma*: connectivity gives every vertex a parent and the count makes the parent map onto, which is all the argument needs. Mathematically this is no surprise—with $n - 1$ edges, connectivity and acyclicity are equivalent characterizations of the same trees—but in a formal development the choice of which characterization to consume is real work saved, and here the acyclic half never has to be materialized.

8 Numerical verification

Each link of the chain was validated numerically—in exact arithmetic, never floating-point—*before* its Lean proof was attempted, and the assembled statement was validated end-to-end afterwards. Table 4 summarizes.

check	scope	result
Cauchy–Binet statement, sorted-subset indexing	$2 \times 3 \cdot 3 \times 2$, exact	match (−264)
tree minors: triangular under the key, $\det = \pm 1$	200 random trees, $n \leq 9$	all pass
non-tree $(n-1)$ -edge subsets: $\det = 0$	158 random graphs	all pass
$(\det N_{0,S})^2 = \mathbb{K}[S \text{ spans a tree}]$, every S	40 random graphs, all S	all pass
$\sum_S (\det N_{0,S})^2 = \det L_0 = \tau(G)$	same 40 graphs	all pass
named graphs (Table 1)	6 graphs	all match
<code>#print axioms</code> , all five theorems	—	3 standard axioms
full library build	lake build	3074 jobs, no errors

Table 4: The verification battery (SageMath 10, exact integer arithmetic).

Size and dependencies. The development is small: 834 lines of Lean across the five files of Table 2, containing 35 declarations in total (28 theorems and lemmas, six definitions, one scoped instance), importing eleven Mathlib files (the `SimpleGraph` core, `LapMatrix`, `IncMatrix`, the graph metric, the determinant and block-triangular API, and `Sym2` ordering). Against a warm Mathlib v4.30 cache each file rechecks in 7–9 seconds; the full project build (library and all parts of the series) is 3074 jobs.

One incident from this battery is worth reporting, because it shows the method earning its keep. The first numerical model of the minor placed ± 1 entries in *every* column, non-edges included, and promptly produced a counterexample to Theorem 5: a spanning star whose edges did not all belong to the graph still had $\det = \pm 1$. The discrepancy was in the model, not the mathematics—the Lean `orientedIncMatrix` gives non-edges *zero* columns, which is precisely case one of the dichotomy—and the corrected model passed all 40 graphs. A verification harness that cannot catch its operator’s own mistakes is decoration; this one caught mine.

9 The boundary: what is proved and what is not

Proved. Theorems 2–1, sorry-free, for finite simple graphs, any root, over any integral domain ($\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$, ...). The five `#print axioms` calls report `propext`, `Classical.choice`, `Quot.sound` and nothing else.

The domain hypothesis. Cases one and three of the dichotomy hold over any commutative ring; the disconnected case turns a kernel vector into a vanishing determinant, which is where zero divisors would interfere. Since the headline equality is between a determinant and a natural number, proving it over $\mathbb{Z}$ (or $\mathbb{Q}$) and transporting is no loss; the hypothesis is stated honestly rather than hidden.

Counting, not constructing. The proof is noncomputable in Lean’s sense: parents are chosen with `Classical.choice`, and no algorithm (for counting or for sampling) is extracted. The determinant on the left is, of course, computable by any linear-algebra system; the theorem is what *justifies* that computation.

Not built, mapped. The weighted version (the Kirchhoff polynomial: give each edge an indeterminate and the reduced determinant enumerates trees by monomials) needs only the same chain with weighted $\pm$ entries—the dichotomy’s triangular argument goes through with x_e in place of ± 1 —but it is not formalized here. Likewise the all-minors theorem [8], the directed/arborescence version [7], and the $\det L_0$ -independence-of-root as a standalone statement (here it is a corollary: both sides count the same trees).

Prior art, stated exactly. For Cauchy–Binet: Mathlib has no version and, at the time of writing, no pull request mentioning it; an independent Lean formalization exists in the project formalizing Grinberg’s *An Introduction to Algebraic Combinatorics* [18, 19], with a different proof architecture (explicit permutation extraction/reconstruction, where this development sums fibers and lets the B -determinant reassemble itself). That project predates this one; the version here is independent and was built for the chain above. For the matrix-tree theorem itself I searched the Lean ecosystem, the Isabelle Archive of Formal Proofs, and the Coq/Rocq corpus, and was unable to locate one; to the best of my knowledge there is none, and I would welcome a correction.

10 Implications and methodology

Where the theorem goes to work. Three consumers, each with the exact quantity the theorem supplies. *Network reliability:* $\tau(G)$ is the all-terminal reliability polynomial’s leading combinatorial datum; comparing designs means comparing spanning-tree counts, and Table 1 is how one computes them. *Random spanning trees:* Wilson’s algorithm [14] samples uniformly among $\tau(G)$ trees; the transfer-current theorem [15] makes edge-membership probabilities determinantal—ratios $\tau(G/e)/\tau(G)$ of tree counts, i.e. effective resistances. *Spectral sparsification:* sampling edges by effective resistance approximately preserves the Laplacian quadratic form, with high probability [16]; the quantity being sampled by is, again, a ratio of the two sides of this theorem. None of these pipelines is formalized here; what is formalized is the identity they all stand on.

For the library. Two pieces are natural Mathlib candidates independently of Kirchhoff: Cauchy–Binet (Section 4), and the oriented incidence matrix with its factorization $NN^\top = D - A$ (Section 5), which complements the existing unoriented `incMatrix` and its $D + A$. The dichotomy argument also isolates a reusable pattern: *replace structural induction by a sort*—find the right injective key, and `BlockTriangular` turns a graph-theoretic recursion into one determinant read-off.

Methodology. As in Parts I–IV, the formalization was carried out with an AI assistant (Claude, Anthropic) operating Lean against a local Mathlib; the human author set the direction, audited every claim, and is responsible for every statement. The working rules are unchanged and brief: “proven” means `sorry`-free with axioms printed; designs are de-risked numerically in exact arithmetic before formalization (Section 8); novelty claims are checked against the literature and the other proof-assistant ecosystems, and prior work found—here, the independent Cauchy–Binet of [19]—is reported rather than elided.

11 Questions left open

A formalization is also a list of the next questions, sharpened until a machine could be asked them.

- The triangular argument reads $\det N_{0,S} = \pm 1$ off a sorted diagonal. The *weighted* minor, with indeterminate x_e in place of ± 1 , is triangular by the same key, with diagonal $\pm \prod_{e \in S} x_e$ (one weight per parent edge). Does the identical proof skeleton yield the Kirchhoff polynomial $\sum_T \prod_{e \in T} x_e$ with only the diagonal read-off changed—and if so, is the weighted statement the *right* Mathlib-level generality from the start?
- The disconnected case needs an integral domain to turn a kernel vector into $\det = 0$. The determinant of a matrix with a rootless component is in fact zero over *any* commutative ring (it is an integer identity). Can the dichotomy be proved ring-generically, e.g. by exhibiting the vanishing as a polynomial identity in the entries, and the domain hypothesis dropped from Theorem 1?
- “Sorting instead of inducting” worked because the parent map plus a linear key made the minor triangular. Which other graph inductions are secretly sorts? The all-minors theorem [8] replaces trees by rooted forests—is there a multi-root key that triangularizes those minors in one pass?
- Both this paper and Part IV ended with a determinant identity whose proof never touches the combinatorial object it counts (there, closed walks; here, cycles). Is there a common formal interface—“count by triangularizing a Gram minor”—under which the matching-side and tree-side determinants become instances of one lemma?
- Mathlib’s `Matrix.TotallyUnimodular` is exactly the predicate “every square minor is 0 or ± 1 ”. The dichotomy establishes this for the *maximal* minors of the reduced matrix N_0 ; the classical fact is that oriented incidence matrices are totally unimodular, all orders included. Would proving and registering full total unimodularity connect this development to the optimization-flavoured part of the library, and does the parent-edge argument extend below the top order?

And one to keep. *Kirchhoff wanted currents, and the determinant handed him the trees he had not asked for; the proof above never once looks at a cycle, yet ends knowing exactly how many ways the graph avoids them all. How much else does a library that can take determinants already know, and simply not yet say?*

References

- [1] J. P. M. Binet, *Mémoire sur un système de formules analytiques, et leur application à des considérations géométriques*, J. Éc. Polytech. **9**, cahier 16 (1813). Read 30 November 1812.
- [2] A.-L. Cauchy, *Mémoire sur les fonctions qui ne peuvent obtenir que deux valeurs égales et de signes contraires par suite des transpositions opérées entre les variables qu’elles renferment*, J. Éc. Polytech. **10**, cahier 17 (1815), 29–112. Read 30 November 1812.
- [3] G. Kirchhoff, *Über die Auflösung der Gleichungen, auf welche man bei der Untersuchung der linearen Vertheilung galvanischer Ströme geführt wird*, Ann. Phys. Chem. **72** (1847), 497–508.

- [4] C. W. Borchardt, *Über eine Interpolationsformel für eine Art symmetrischer Functionen und über deren Anwendung*, Abh. Königl. Akad. Wiss. Berlin (1860), 1–20.
- [5] A. Cayley, *A theorem on trees*, Quart. J. Pure Appl. Math. **23** (1889), 376–378.
- [6] R. L. Brooks, C. A. B. Smith, A. H. Stone, and W. T. Tutte, *The dissection of rectangles into squares*, Duke Math. J. **7** (1940), 312–340.
- [7] W. T. Tutte, *The dissection of equilateral triangles into equilateral triangles*, Proc. Cambridge Philos. Soc. **44** (1948), 463–482.
- [8] S. Chaiken and D. J. Kleitman, *Matrix tree theorems*, J. Combin. Theory Ser. A **24** (1978), 377–381.
- [9] S. Chaiken, *A combinatorial proof of the all minors matrix tree theorem*, SIAM J. Algebraic Discrete Methods **3** (1982), 319–329.
- [10] J. W. Moon, *Counting Labelled Trees*, Canadian Math. Monographs **1**, Canadian Math. Congress, 1970.
- [11] N. Biggs, *Algebraic Graph Theory*, 2nd ed., Cambridge Univ. Press, 1993.
- [12] C. Godsil and G. Royle, *Algebraic Graph Theory*, Grad. Texts in Math. **207**, Springer, 2001.
- [13] R. P. Stanley, *Enumerative Combinatorics, Volume 2*, Cambridge Stud. Adv. Math. **62**, Cambridge Univ. Press, 1999.
- [14] D. B. Wilson, *Generating random spanning trees more quickly than the cover time*, in *Proc. 28th ACM Symp. Theory of Computing* (STOC 1996), 296–303.
- [15] R. Burton and R. Pemantle, *Local characteristics, entropy and limit theorems for spanning trees and domino tilings via transfer-impedances*, Ann. Probab. **21** (1993), 1329–1371.
- [16] D. A. Spielman and N. Srivastava, *Graph sparsification by effective resistances*, SIAM J. Comput. **40** (2011), 1913–1926.
- [17] R. Lyons and Y. Peres, *Probability on Trees and Networks*, Cambridge Univ. Press, 2016.
- [18] D. Grinberg, *An Introduction to Algebraic Combinatorics*, lecture notes, 2026. [arXiv:2506.00738](https://arxiv.org/abs/2506.00738).
- [19] *algebraic-combinatorics: a Lean 4 formalization of Grinberg’s “An Introduction to Algebraic Combinatorics”*, <https://github.com/faabian/algebraic-combinatorics>, 2026. Cauchy–Binet in blueprint §5.2.
- [20] The mathlib Community, *The Lean mathematical library*, in *Proc. 9th ACM SIGPLAN Int. Conf. on Certified Programs and Proofs* (CPP 2020), 367–381.
- [21] C. Marín, *Random Signs into Matchings: the Godsil–Gutman estimator and real-rootedness in Lean 4* (Part I), 2026. Zenodo, DOI [10.5281/zenodo.20517350](https://doi.org/10.5281/zenodo.20517350).
- [22] C. Marín, *Unfolding a Graph into a Tree: A Machine-Checked Proof of the Heilmann–Lieb Theorem in Lean 4* (Part II), 2026. Zenodo, DOI [10.5281/zenodo.20561832](https://doi.org/10.5281/zenodo.20561832).

- [23] C. Marín, *Folding Edges into Vertices: A Machine-Checked Proof of Bass's Determinant Formula for the Ihara Zeta in Lean 4*, 2026. Zenodo, DOI 10.5281/zenodo.20573120.
- [24] C. Marín, *Walks that Forget the Cycles: A Machine-Checked Bijection between Tree-Like Walks of a Graph and Walks on Godsil's Path Tree, in Lean 4* (Part III), 2026. Zenodo, DOI 10.5281/zenodo.20600326.
- [25] C. Marín, *When Walks Become a Spectrum: A Machine-Checked Proof of Godsil's Moment Theorem in Lean 4* (Part IV), 2026. Zenodo, DOI 10.5281/zenodo.20613247.